SDBM REPORT

Lucía Jiménez, Emma Rodríguez and Álvaro Martín Ruiz

100472272 - 100475225 - 100475318

Group R11

INDEX

BLOCK DIAGRAM AND SCHEMATICS OF CONNECTIONS..... 3

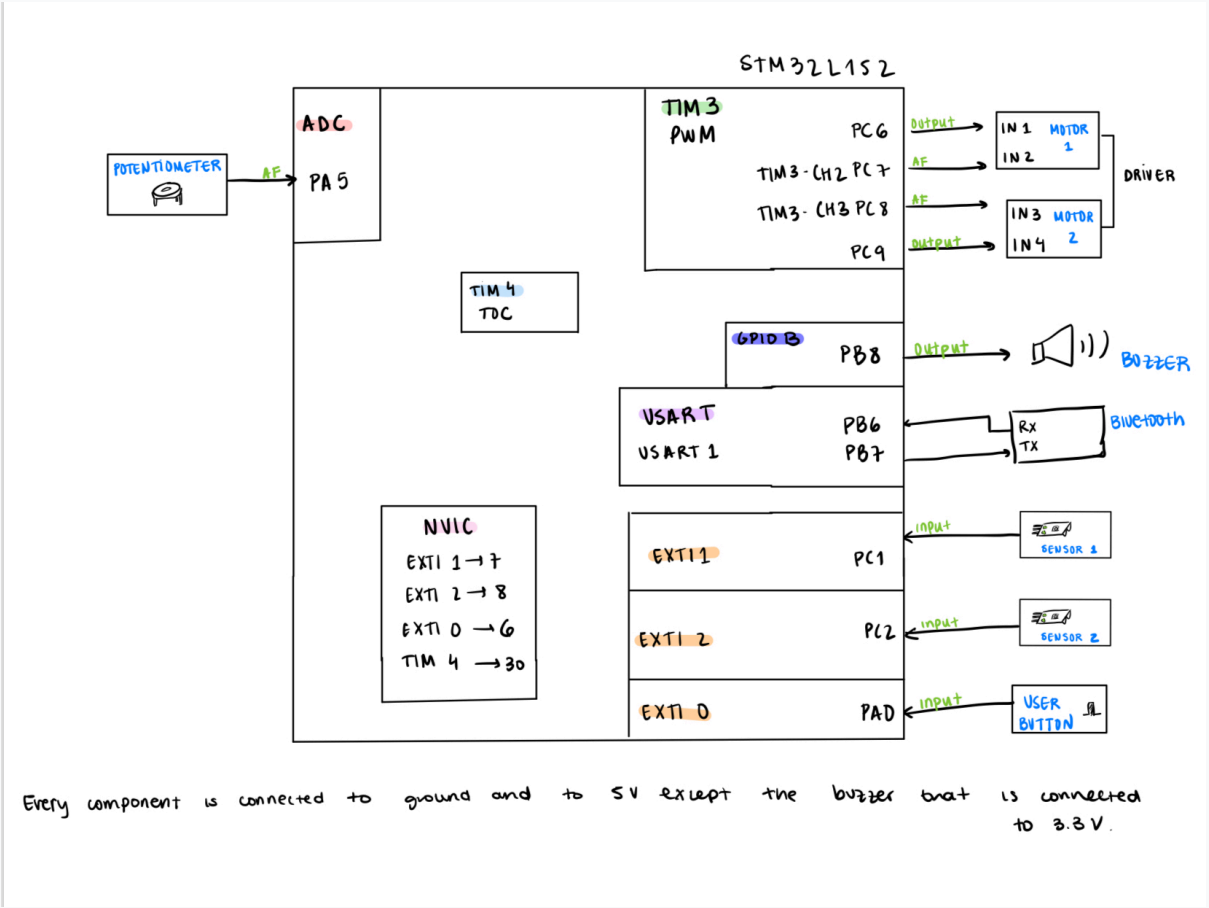
EXPLANATION OF THE PERIPHERALS OF THE MICROCONTROLLER.....4

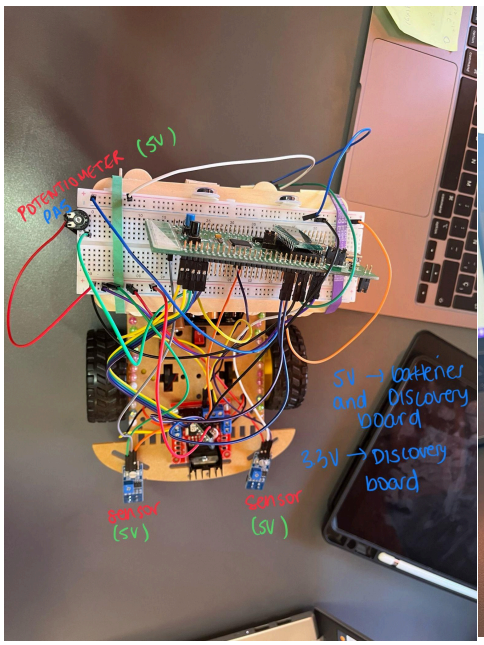
FLOWCHARTS OF THE ISRs DEVELOPED AND RELEVANT FUNCTIONS..... 6

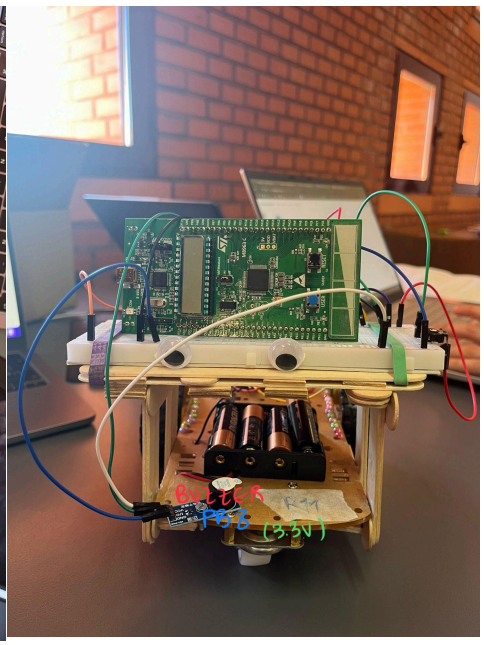
EXPLANATION OF THE IRQ..... 14

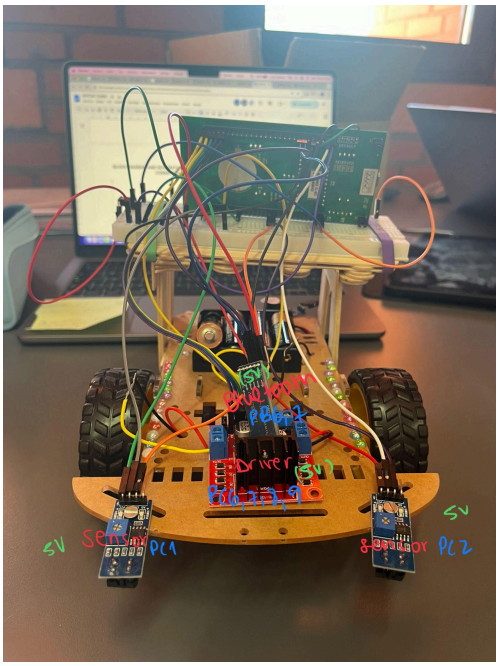
CONCLUSION..... 16

BLOCK DIAGRAM AND SCHEMATICS OF CONNECTIONS









EXPLANATION OF THE PERIPHERALS OF THE MICROCONTROLLER

WHEELS

The wheels are connected to the driver which are connected to the IN1, IN2, IN3, IN4 of the drivers which are connected to the PC6, PC7, PC8, PC9 pins.
PC7 and PC8 are configured as alternate function (AF) so they are connected to the PWM of TIM3 so we can control velocity. TIM3 has channel 2 and channel 3 associated with these pins, to do this we use the AFR to associate to TIM3.
PC6 and PC9 are configured as output.

SENSORS

The sensors are connected to pins PC1 and PC2. These are configured as inputs. If they detect black the ISR of those pins is written to 1. If they detect white they write a 0. Because of this they are configured as falling and rising edges. They activate the EXTI 1 and EXTI2, which have the interrupts active. With these we decide which direction the car should take.

BUZZER

The buzzer is connected to PB8 in output mode. According to the states that we read from the sensors, the TIM4 handler changes when the buzzer is on, off or intermittent. It is not connected to the TIM4 explicitly, but is done in this handler to control the intermittent mode.

POTENTIOMETER

The potentiometer is connected to PA5 in analog function mode. It is connected to the ADC. We configure the ADC to one single conversion using channel 4. We use continuous conversion. The potentiometer is used to change max and min velocities of the wheels.

BLUETOOTH

The bluetooth is connected to PB6 and PB7. PB6 is connected to the Rx and PB7 to the Tx. These are in charge of the USART communication, we use interrupts for both of them, to receive and transmit. The bluetooth is used to give orders from the mobile phone to the car, and to transmit the messages of what is going on in the car. eg going forward, backwards etc.

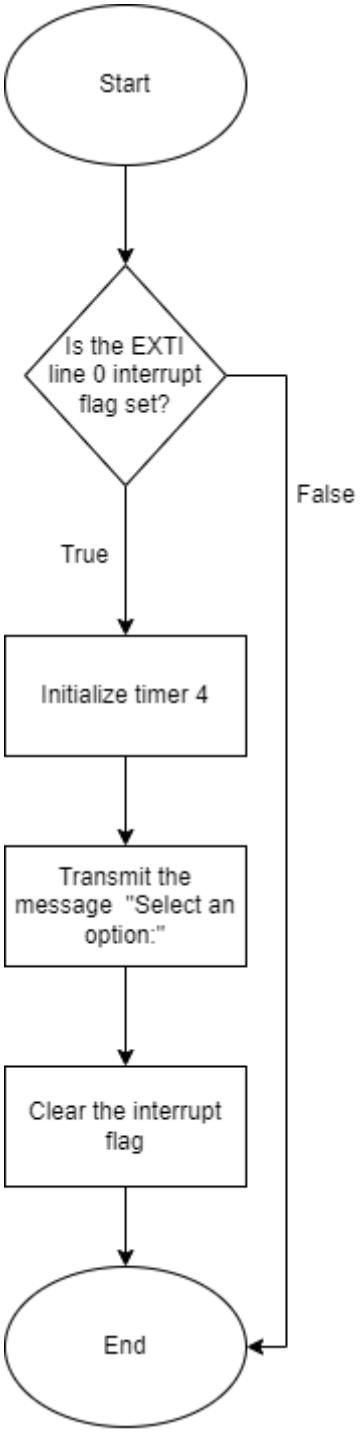
USER BUTTON

The user button is connected to PA0 as a digital input. The EXTI0 is the one associated with this pin, and it's configured to use interrupts and only for falling edges.
When the button is pressed, the TIM4 is initialized, and waits 2 seconds (established in CCR1) before starting to move, receive and send information.

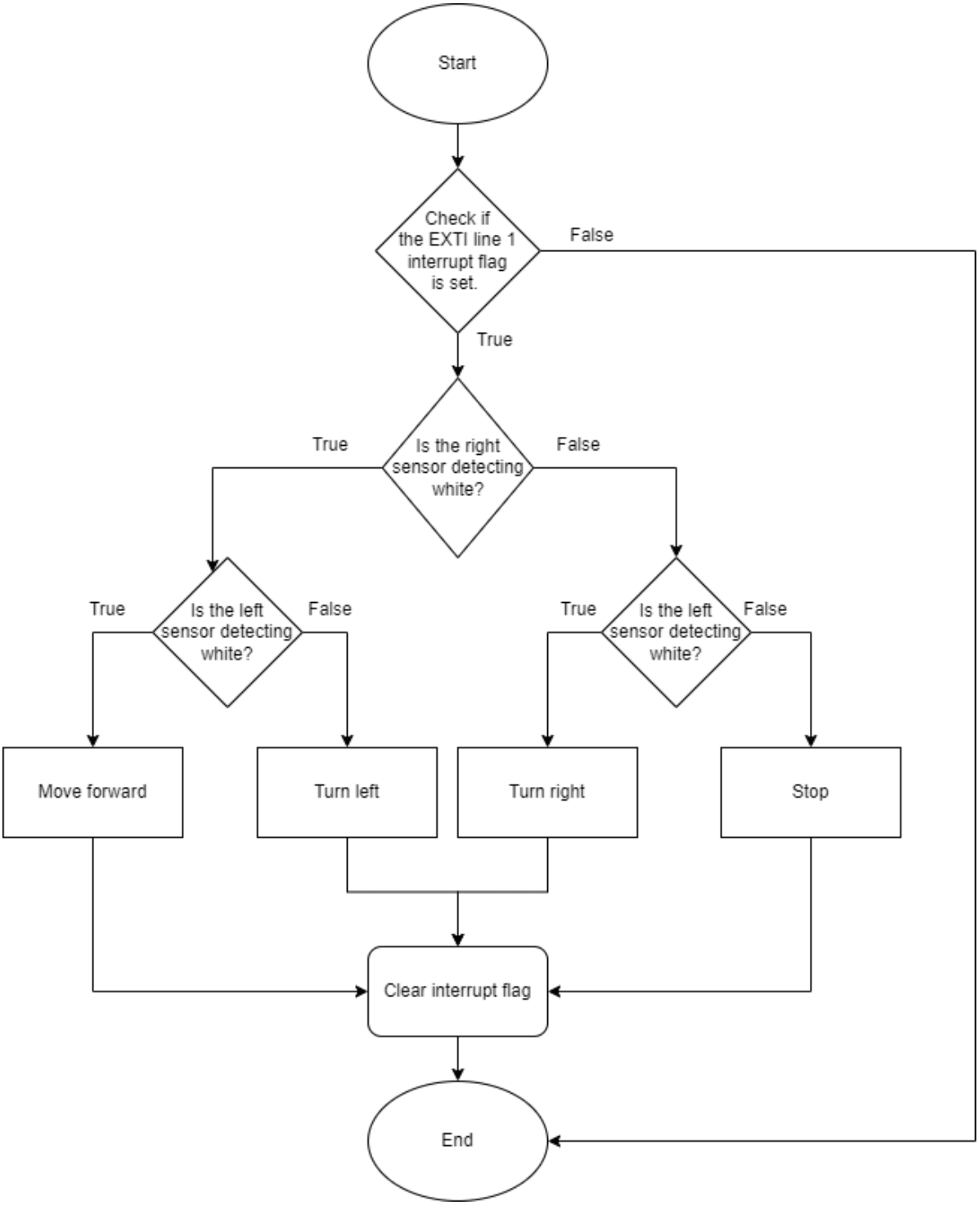
FLOWCHARTS OF THE ISRs DEVELOPED AND RELEVANT FUNCTIONS

Flowcharts have to be descriptive, not including specific microcontroller hardware details.

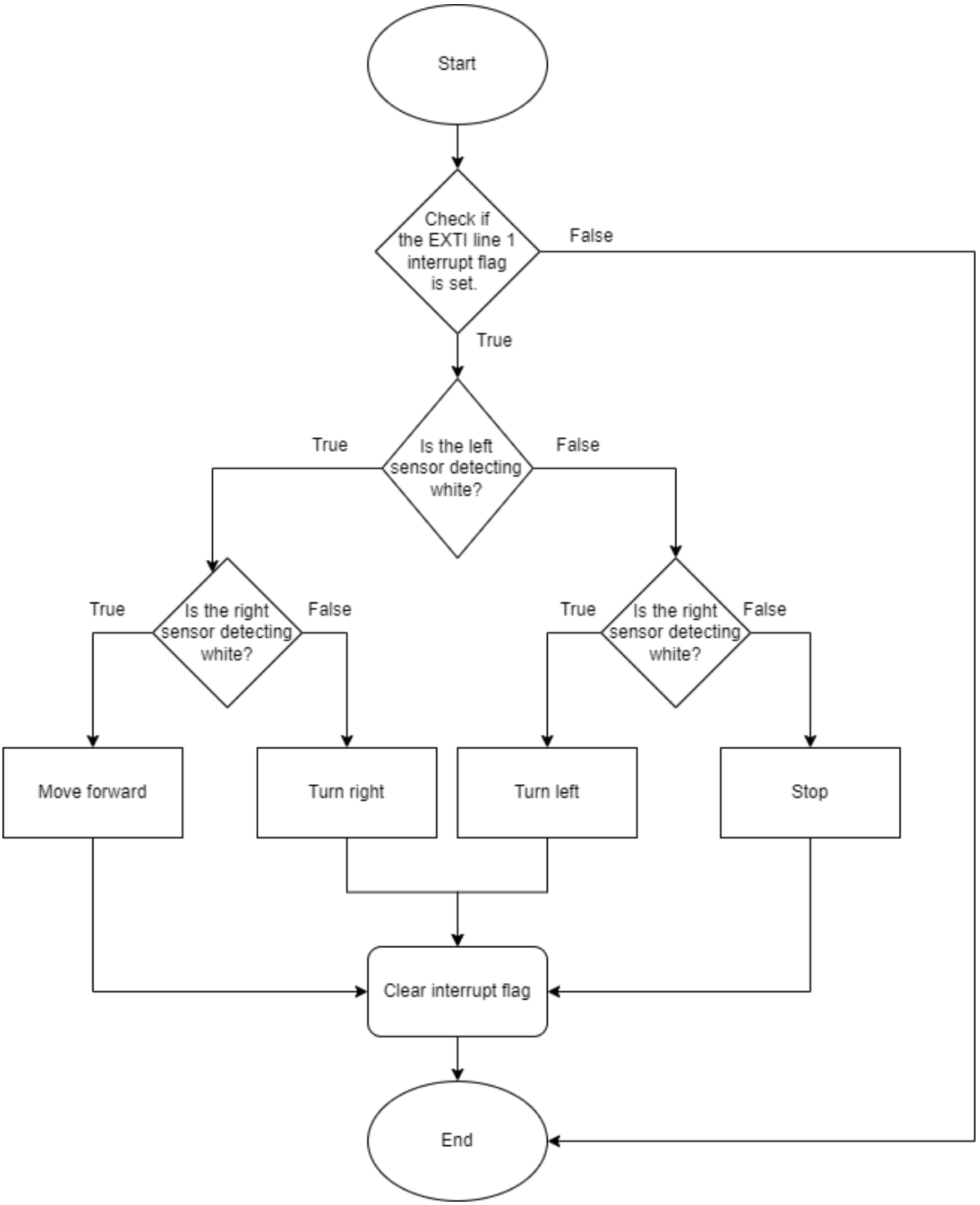
- **EXTI0_IRQHandler** (user button)



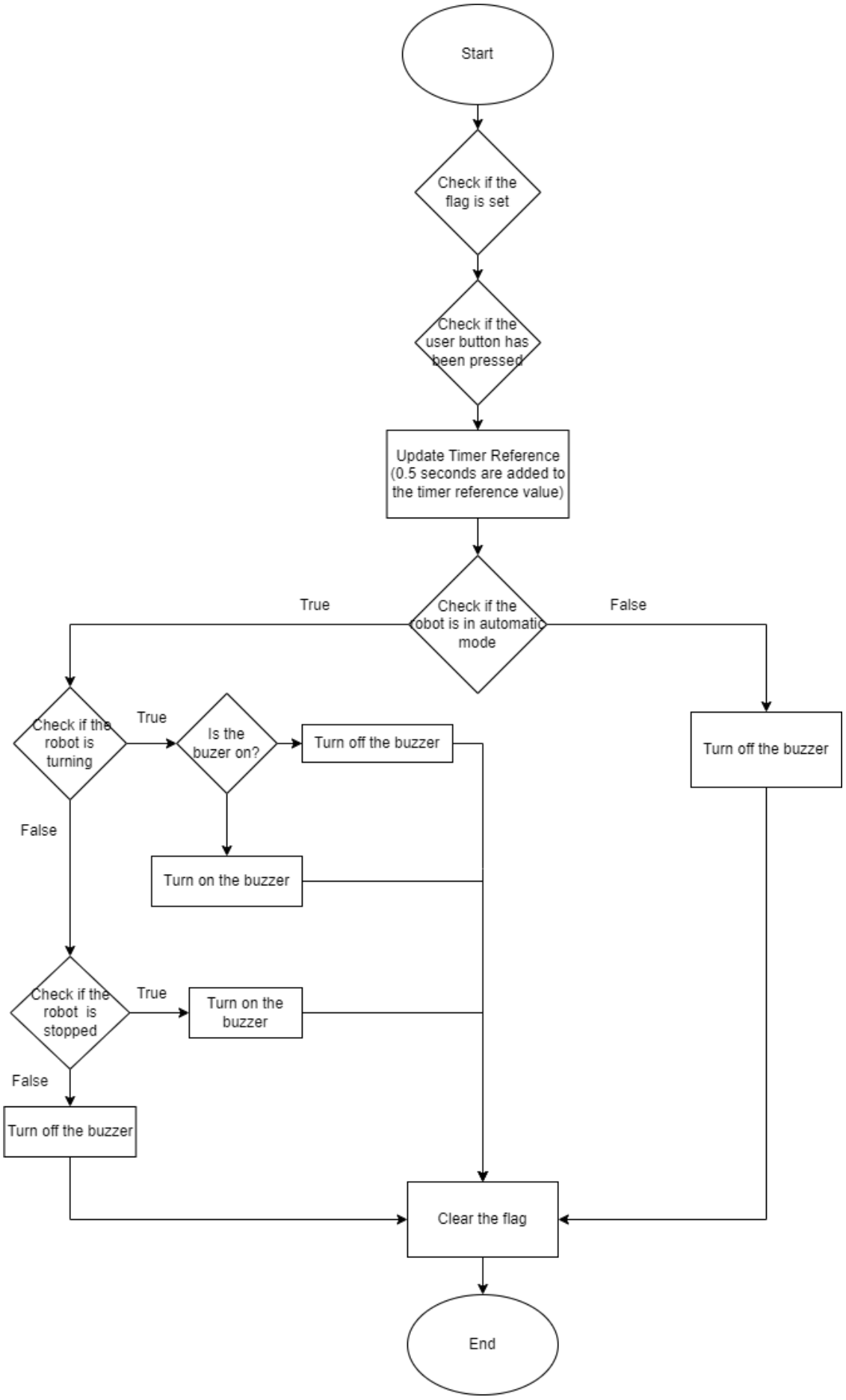
- **EXTI1_IRQHandler** (right infrared sensor)



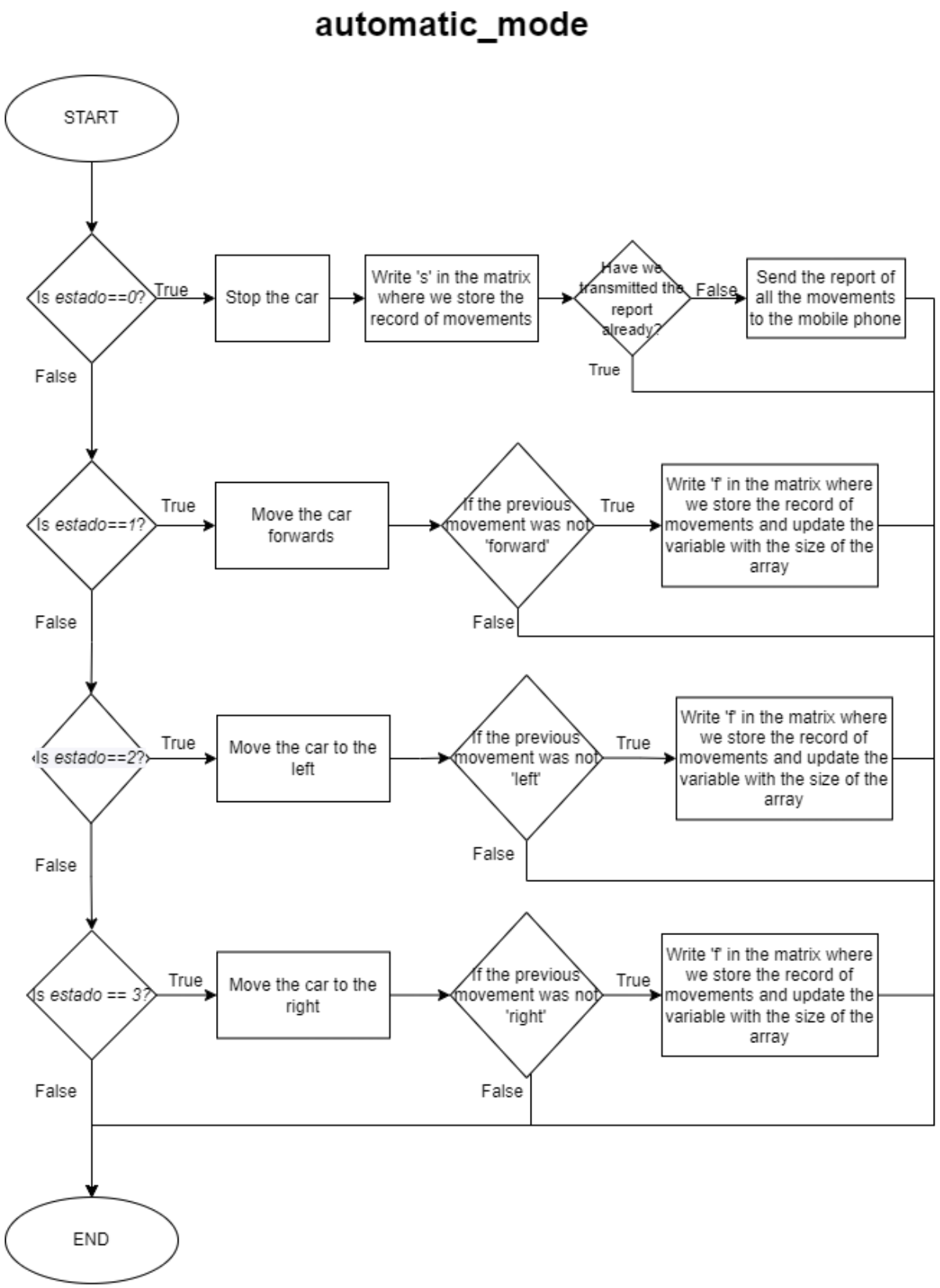
- **EXTI2_IRQHandler** (left infrared sensor)



- **TIM4_IRQHandler** (buzzer)

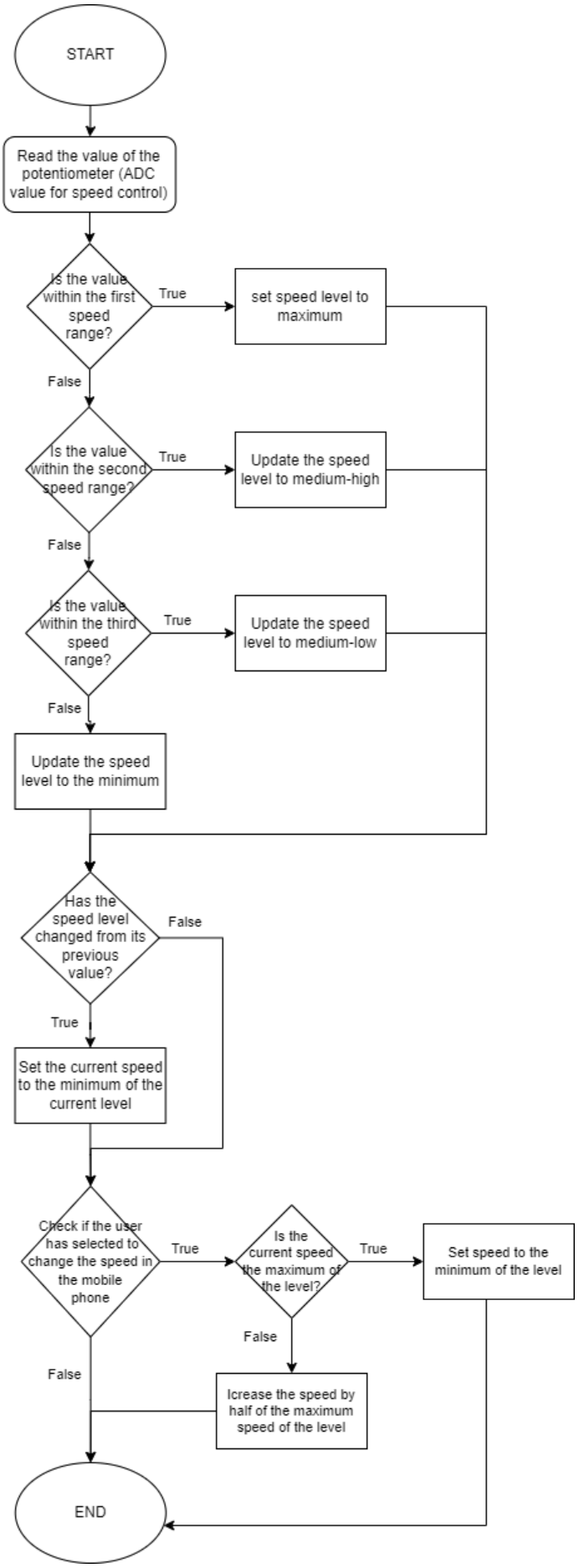


- **Function for the automatic mode function**

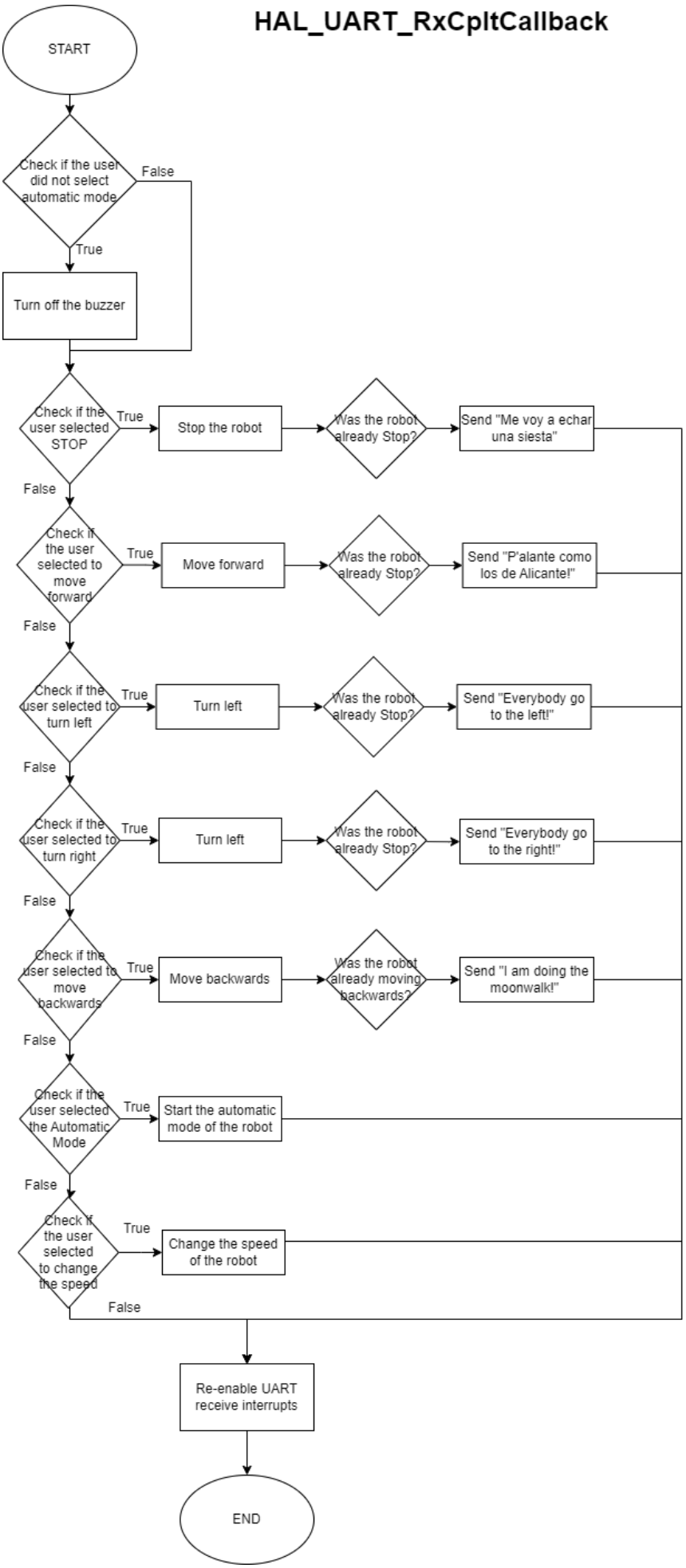


- **Function to control the speed**

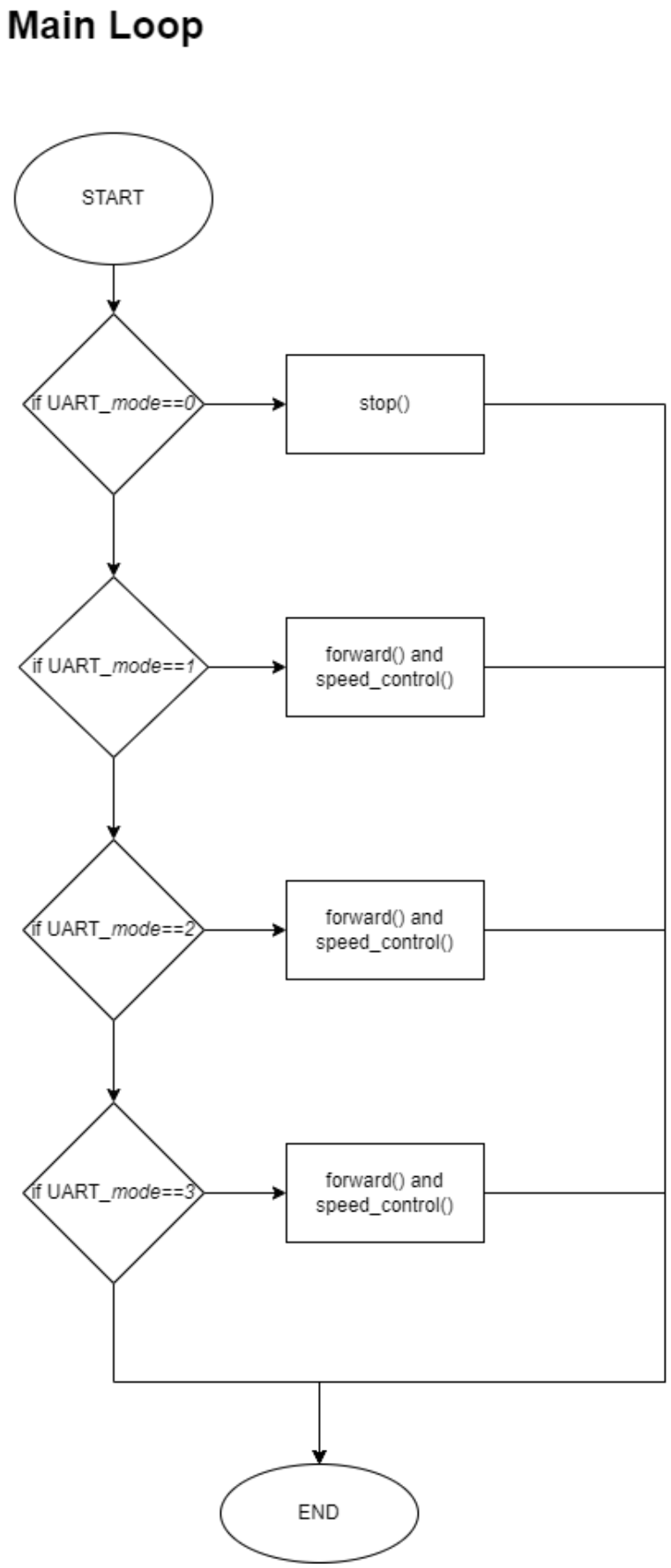
speed_control



- **Callback function**



- **Main loop**


Main Loop
START
if UART_mode==0
stop()
if UART_mode==1
forward() and speed_control()
if UART_mode==2
forward() and speed_control()
if UART_mode==3
forward() and speed_control()
END

EXPLANATION OF THE IRQ

Which IRQ have you used and the functionality achieved by their ISRs

1. **EXTI0_IRQHandler**

Triggered by a press of the USER button.
It Initializes Timer 4. The interrupt of the timer 4 will be triggered 2 seconds after the user button is pressed (this is achieved because Timer 4 is set to count 2 seconds before triggering its interrupt).
We transmit the message “Select an option: ” to the mobile phone (the device connected)
Clear the flag.

2. **EXTI1_IRQHandler & EXTI2_IRQHandler**

Triggered by a rising or falling edge detected by one of the IR sensors (EXTI1 is associated with the right IR sensor, and EXTI2 with the left IR sensor), indicating a change in the detected surface (black to white and vice versa).
Checks the state of two IR sensors to determine if the robot is currently detecting a straight line (white-white), a turn (white-black or black-white) or the end of the circuit (black-black).
Based on the combination of IR sensor readings, it decides the movement of the robot, which can be forward, right turn, left turn, or stop (there's a state machine that decides the next action based on the combination of sensor readings).
Clear the flag.

3. **TIM4_IRQHandler**

Activated when Timer 4 matches a reference value.
Checks if it is the first iteration in this handler function (if button_pressed ==0). If it is the first one, we update the variable button_pressed =1. This variable is used in our code to know if the robot has been turned on (i.e., if the user button has been pressed).
Adds 0.5 seconds to the reference value for the timer, adjusting when the interrupt will trigger again. The value of 0.5 seconds has been chosen in order to be able to generate a toggle effect in the buzzer when needed (turning it on/off every 0.5 seconds).

If the robot is in automatic mode, we differentiate between the following scenarios to decide whether to turn on or off the buzzer.

- If going forward, the buzzer is turned off.
- If stopped, the buzzer is turned on.
- If turning, the buzzer state is toggled (turned on if it was off and vice versa).

We can see the current move of the robot using the variable estado, which is a 0 if it is stopped, a 1 if it is moving forward, a 2/3 if it is moving right/left The variable UART_mode informe us if the robot is in manual or automatic mode or if a change of speed was selected.
In manual mode, the buzzer is turned off.

4. **HAL_UART_RxCpltCallback** (although not an IRQ itself, it is a callback related to UART interrupts)

It is called when a UART receive interrupt is completed. Its primary role is to process incoming commands, which are received as single characters from the UART peripheral. Each character corresponds to a specific command that alters the state or behavior of the robot.

It processes the received commands and adjusts the robot's operation mode accordingly (stop, move forward, backward, etc.). This is done by checking the character received from the UART peripheral and set the UART_mode variable accordingly in the following way:

- '0' sets the robot to stop.
- '1' commands the robot to move forward.
- '2' commands the robot to turn left.
- '3' commands the robot to turn right.
- '4' commands the robot to move backward.
- '5' puts the robot into automatic mode.
- '6' signals that the robot should change speed.

Also, If the mode has changed from the previous state (*previous_movement*), a message informing of the new movement is sent to the connected device.
Re-enable UART Reception.

CONCLUSION

Overall, this project was as intense as it was educational. We nailed all the main goals for our robot—it now follows lines, responds to sensors, buzzes when necessary, and obeys every command from our phones. We didn't tackle the extra requirements, but that was just down to running out of time, not a lack of ambition.

When it comes to the effort we put in, this was way more than just coding and assembling. There were a few stressful moments where we couldn't figure out why things weren’t working as expected. We poured a ton of time into this, but seeing our robot navigate as we envisioned made it all worthwhile in the end.

Regarding what we have learned, we can firmly say that we have acquired a tone of valuable knowledge, the kind that you can't get from just doing exams. This project was the type of challenge that spurs growth and learning.

After all the hours we dedicated, we're walking away with much more than a functioning line-following robot. We’ve acquired a toolbox of hands-on experience and a deepened understanding of the concepts we put into practice.